Welcome to the documentation of singletCode!

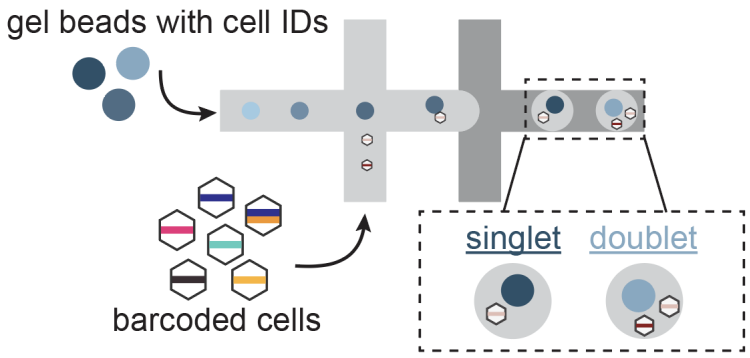
singletCode (link to preprint) uses cell barcoding technology to identify **true singlets** in scRNAseq data. In our preprint, we used singletCode to benchmark existing doublet detection methods and train a classifier to detect singlets in non-barcoded datasets as well of the same cell type.

doublets in sequencing

- How do doublets form?
- Downstream impact of doublets

How do doublets form?

Single-cell RNA sequencing (scRNA-seq) datasets contain true single cells, or singlets, in addition to cells that randomly coalesce during the protocol, or doublets. Sometimes, there are higher rates of doublets which can be attributed to cellular physiology and experimental protocols can lead to cell clumping. Doublet percentage in a sample be as high as 40%. Doublets can be two very transcriptionally different cells captured together (heterotypic) or two transcriptionally similar cells captured together (homotypic). Doublets are difficult to identify because just because two cells are captured together does not mean there is simply more absolute RNA fragments present or sequenced in doublet cases, and cells exist on a transcriptional continuum, making identification of valid singlet cells difficult, especially if cells are in a transitioning or reprogrammed state.

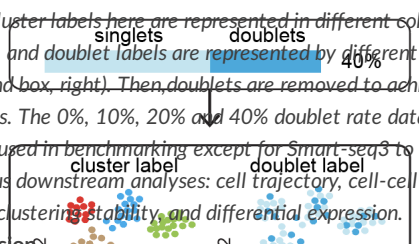


Schematic of how cells with lineage barcodes appear in the single-cell sequencer where droplets add a unique cell ID. The singlet depicted is one reaction droplet with a single cell such that there is a 1:1:1 mapping of a cell to a barcode to a cell ID. Alternatively, an example of a doublet is when one droplet has two or more cells, each having a unique barcode.

Downstream impact of doublets

Doublets are problematic because they can impact the conclusions of scRNA-seq downstream functional analyses. We found that doublets confound downstream analyses when we tested common downstream scRNA-seq data analysis protocols (differential expression, cell trajectory, clustering stability, and cell-cell communication) on datasets of various doublet percentages. A schematic for how these datasets were generated and examples of our results for differential expression are below:

Schematic of datasets generated for all functional analyses. The dataset is first processed to have 40% doublets and undergo standard Seurat processing with Leiden clustering, where cluster labels here are represented in different colors in UMAP space (second box, left), and doublet labels are represented by different shades of blue in UMAP space (second box, right). Then,doublets are removed to achieve 20%, 10% and 0% doublet datasets. The 0%, 10%, 20% and 40% doublet rate datasets are generated for each dataset used in benchmarking except for Smart-seq3 to evaluate doublet impact on various downstream analyses: cell trajectory, cell-cell communication, clustering stability, and differential expression.



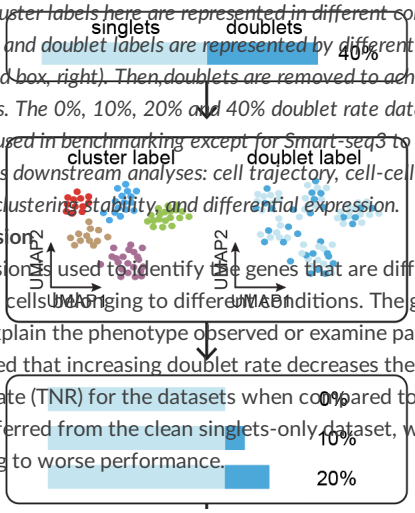
differential expression

Schematic of how cells with lineage barcodes appear in the single cell sequencer. Where droplets add a unique cell ID. The singlet depicted is one reaction droplet with a single cell such that there is a 1:1:1 mapping of a cell to a barcode to a cell ID. Alternatively, an example of a doublet is when one droplet has two or more cells, each having a unique barcode.

Downstream impact of doublets

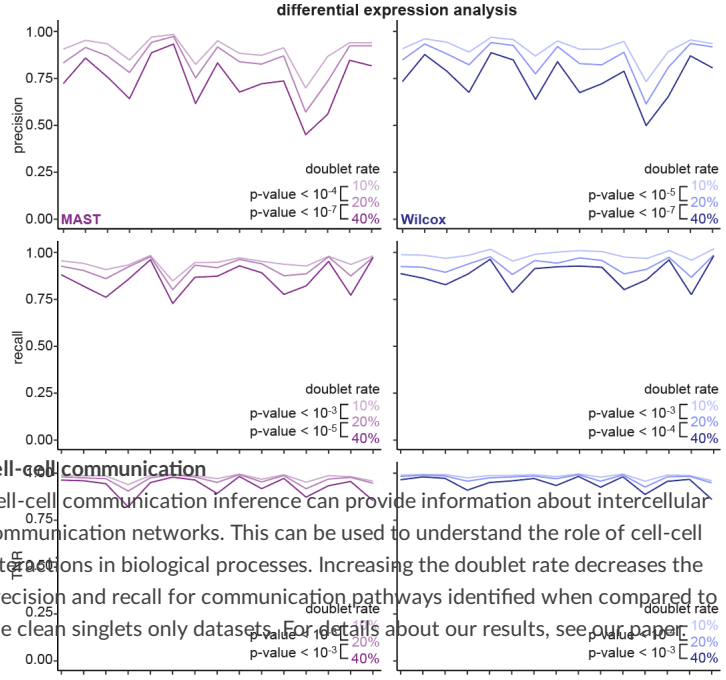
Doublets are problematic because they can impact the conclusions of scRNA-seq downstream functional analyses. We found that doublets confound downstream analyses when we tested common downstream scRNA-seq data analysis protocols (differential expression, cell trajectory, clustering stability, and cell-cell communication) on datasets of various doublet percentages. A schematic for how these datasets were generated and examples of our results for differential expression are below.

Schematic of datasets generated for all functional analyses. The dataset is first processed to have 40% doublets and undergo standard Seurat processing with Leiden clustering, where cluster labels here are represented in different colors in UMAP space (second box, left), and doublet labels are represented by different shades of blue in UMAP space (second box, right). Then,doublets are removed to achieve 20%, 10% and 0% doublet datasets. The 0%, 10%, 20% and 40% doublet rate datasets are generated for each dataset used in benchmarking except for Smart-seq3 to evaluate doublet impact on various downstream analyses: cell trajectory, cell-cell communication, clustering stability, and differential expression.



differential expression

Differential expression is used to identify the genes that are differentially expressed between cells belonging to different conditions. The genes identified could be used to explain the phenotype observed or examine pathways involved. Our results indicated that increasing doublet rate decreases the precision, recall, and true negative rate (TNR) for the datasets when compared to the differentially expressed genes inferred from the clean singlets-only dataset, with a higher doublet rate leading to worse performance.



Differential expression analysis results for all datasets using MAST (purple, left) and Wilcox (blue, right) on two randomly chosen clusters at various doublet rates. Line color shade represents the % doublet rate, with a more saturated color indicative of a higher doublet rate. Precision, recall, and TNR are calculated against ground truth differentially expressed genes in the corresponding dataset without doublets. P-values between variable doublet rate datasets are calculated with a paired Wilcoxon signed-rank test.

cell-cell communication

Cell-cell communication inference can provide information about intercellular communication networks. This can be used to understand the role of cell-cell interactions in biological processes. Increasing the doublet rate decreases the precision and recall for communication pathways identified when compared to the clean singlets only datasets. For details about our results, see our paper.

cell trajectory

Increasing doublet rate caused the trajectory to deviate away from the trajectory inferred from the clean singlet only datasets and there were gain/loss of lineages when doublets were introduced. For details about our results, see our paper.

clustering stability

Clustering is one of the most common analyses done to infer the identity of similar cells. When the number of doublets were increased, the probability of getting the correct number of cell clusters decreased. This might lead to spurious clusters or a genuine sub-type of cells not being identified as a distinct group. For details about our results, see our paper.

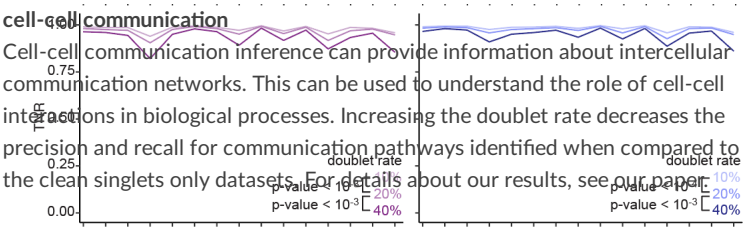
what is singletCode?

singletCode is a framework to extract true singlets from barcoded scRNA-seq data. Our pipeline identified barcoded singlets from 10 different publications and 6 original experimental scRNA-seq samples generated in this study, encompassing 7 different barcoding technologies, 94 scRNA-seq samples, 3 unique sequencing technologies, and a total of 364,579 cells. Of the 338,948 barcoded cells, we extracted 293,618 singlets.(read more about the barcoded datasets we used singletCode with here).

singlets | doublets

cell-cell communication

Cell-cell communication inference can provide information about intercellular communication networks. This can be used to understand the role of cell-cell interactions in biological processes. Increasing the doublet rate decreases the precision and recall for communication pathways identified when compared to the clean singlets only datasets. For details about our results, see our paper.



Differential expression analysis results for all datasets using MAST (purple, left) and Wilcox (blue, right) on two randomly chosen clusters at various doublet rates. Line color shade represents the % doublet rate, with a more saturated color indicative of a higher doublet rate. Precision, recall and TNR are calculated against ground truth differentially expressed genes in the corresponding dataset without doublets. P-values between variable doublet rate datasets are calculated with a paired Wilcoxon signed-rank test.

cell trajectory

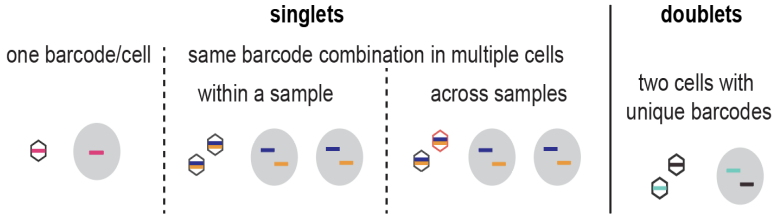
Increasing doublet rate caused the trajectory to deviate away from the trajectory inferred from the clean singlet only datasets and there were gain/loss of lineages when doublets were introduced. For details about our results, see our paper.

clustering stability

Clustering is one of the most common analyses done to infer the identity of similar cells. When the number of doublets were increased, the probability of getting the correct number of cell clusters decreased. This might lead to spurious clusters or a genuine sub-type of cells not being identified as a distinct group. For details about our results, see our paper.

what is singletCode?

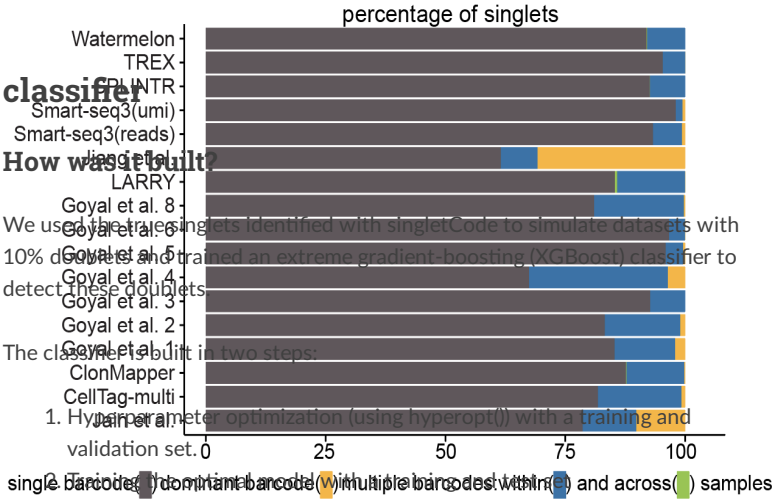
singletCode is a framework to extract true singlets from barcoded scRNA-seq data. Our pipeline identified barcoded singlets from 10 different publications and 6 original experimental scRNA-seq samples generated in this study, encompassing 7 different barcoding technologies, 94 scRNA-seq samples, 3 unique sequencing technologies, and a total of 564,579 cells. Of the 338,948 barcoded cells, we extracted 293,618 singlets.(read more about the barcoded datasets we used singletCode with here).



Singlet identification

singletCode leverages the fact that the lineage barcodes are present within the cell before sequencing. As a result of multiplicity of infection, singletCode identifies cells which meet any of the following conditions as a true singlet:

1. 1 barcode/1 cell ID
2. >1 barcode per 1 cell ID, but 1 barcode has significantly more UMI counts than the other barcodes within the same cell
3. M barcodes per 1 cell ID, but the same combination of M barcodes are found in other cells in the same sample
4. M barcodes per 1 cell ID, but the same combination of M barcodes are found in other cells across samples within the same experimental design (common in barcoding studies)



Stacked bar chart showing types of singlets in each dataset. Each color represents one type of one cell ID has has the highest scenarios where across twin

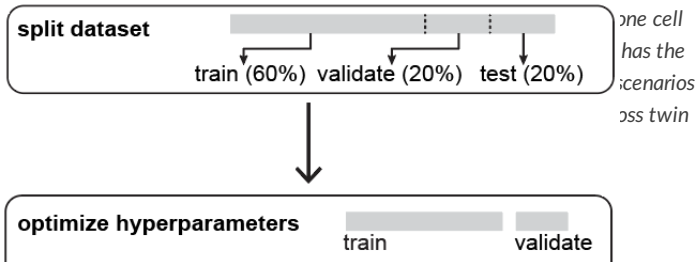
classifier

How was it built?

We used the true singlets identified with singletCode to simulate datasets with 10% doublets and trained an extreme gradient-boosting (XGBoost) classifier to detect these doublets.

The classifier is built in two steps:

1. Hyperparameter optimization (using hyperopt()) with a training and validation set.
2. Training the optimal model with training and validation sets.



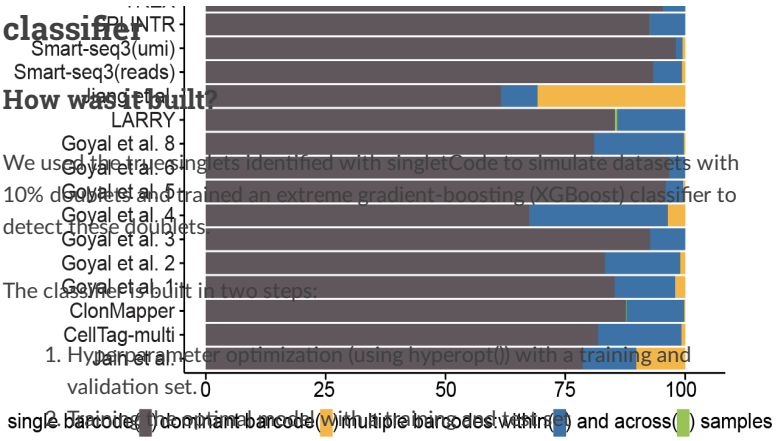
classifier

How was it built?

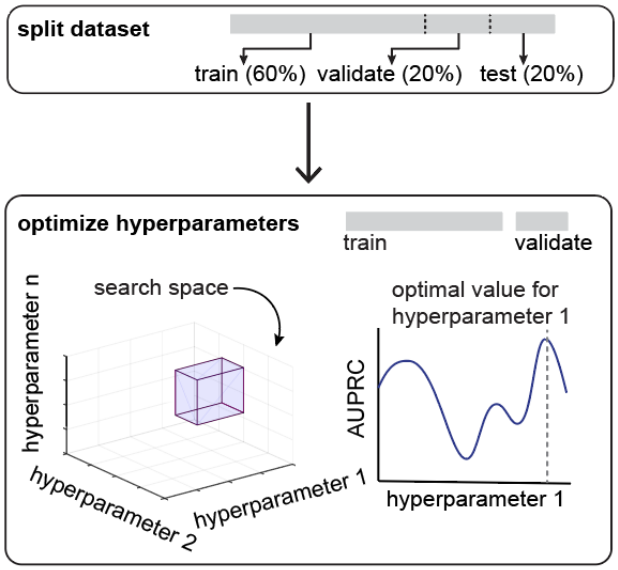
We used the true singlets identified with singletCode to simulate datasets with 10% doublets and trained an extreme gradient-boosting (XGBoost) classifier to detect these doublets.

The classifier is built in two steps:

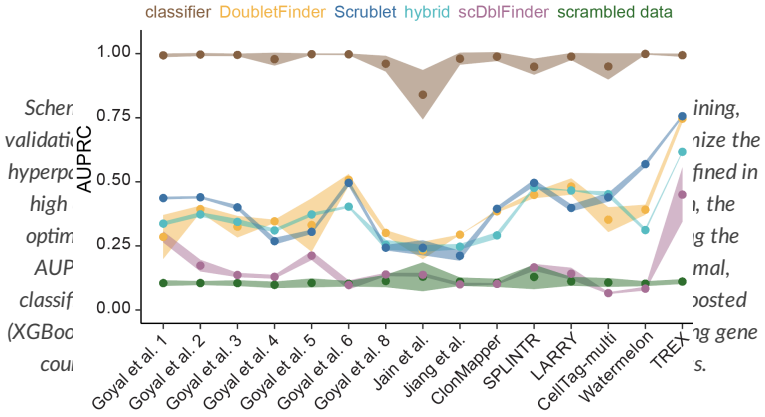
1. Hyperparameter optimization (using hyperopt()) with a training and validation set.
2. Training the optimal model with training and validation sets.



Stacked bar chart showing types of singlets in each dataset. Each color represents one type of cell ID has the highest scenarios where cross twin



*Scheme of training, validation and optimization of the hyperparameters used in the classifier. (XGBoost) classifier. The classifier uses the AUPRC score to find the optimal, boosted



Scher ... ining, validati ... nize the hyperp ... fined in high ... , the optim ... g the AUP ... mal, classif ... osted (XGBoo ... ng gene cou ... s.

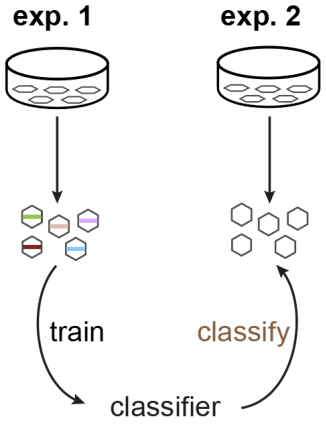
How well does it perform?

Results of classifier performance compared to other doublet detection methods as measured by AUPRC. Each dot represents the average AUPRC score of a doublet detection method on a given dataset (x-axis). The ribbon surrounding the dot represents the standard deviation of the AUPRC score for each doublet detection method.

We achieved significantly higher AUPRC and AUROC scores using our classifier compared to other methods. We also ranked first for doublet detection.

Classifying doublets in non-barcoded datasets

Although barcoding experiments are becoming increasingly prevalent, they are still relatively uncommon. Therefore, we sought to train a doublet classifier on barcoded data that could detect doublets in non-barcoded data. We trained a classifier on cell samples from melanoma, mouse brain, leukemia, and bone marrow/leukemia using true singlet labels from singletCode and successfully identified doublets in a similar cell sample without needing barcoding data. Further, we integrated all the data from mouse and human samples together, split it in half, and it to train a classifier that was then tested on individual sample components of the opposite half. This, too, proved to outperform other doublet detection methods.



Schematic of training a doublet classifier on barcoded data from 1 experiment and using that classifier to identify doublets in a biological replicate, experiment 2.

Non-barcoded datasets

As barcoded scRNA-seq data becomes more abundant, experimenters can train a classifier specific to their cell type using a barcoded subset of data, and, eventually, there could be enough data to train a classifier on multiple barcoded cell types which can be used more generally.

If the data is non-barcoded, the information from true singlets identified from singletCode can still be harnessed using a classifier trained on barcoded data from a different experiment using the cell type of interest. Find more information about this in the classifier page here.

implementing singletCode on your data benchmarking doublet detection methods

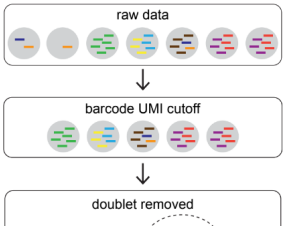
singletCode is a framework that can be extended to most scRNA-seq or even scATAC-seq data to accurately identify singlets.

Simulation datasets

To simulate doublets for benchmarking, we randomly selected the gene expression counts data from two cells that were found to be true singlets by singletCode. We averaged the counts from these two cells to generate simulated doublets and create datasets with various doublet percentages for benchmarking.

Barcoded datasets

We created both a python package called singletCode and a command-line interface. The only input needed is a .csv file with cell ID, barcode, sample information, and UMI counts for each barcode. You can find more information to use it here



that classifier to identify doublets in a biological replicate, experiment 2.

Non-barcoded datasets

If the data is non-barcoded, the information from true singlets identified from singletCode can still be harnessed using a classifier trained on barcoded data from a different experiment using the cell type of interest. Find more information about this in the classifier page here.

As barcoded scRNA-seq data becomes more abundant, experimenters can train a classifier specific to their cell type using a barcoded subset of data, and, eventually, there could be enough data to train a classifier on multiple barcoded cell types which can be used more generally.

implementing singletCode on your data benchmarking doublet detection methods

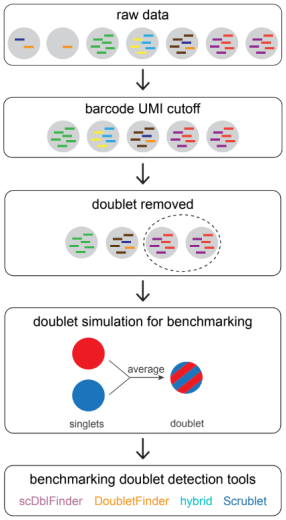
singletCode is a framework that can be extended to most scRNA-seq or even scATAC-seq data to accurately identify singlets.

Simulation datasets

To simulate doublets for benchmarking, we randomly selected the gene expression counts data from two cells that were found to be true singlets by singletCode. We averaged the counts from these two cells to generate simulated doublets and create datasets with various doublet percentages for benchmarking.

Barcoded datasets

We created both a python package called singletCode and a command-line interface. The only input needed is a .csv file with cell ID, barcode, sample information, and UMI counts for each barcode. You can find more information to use it here



Workflow schematic of how raw data is processed to generate true singlets based on a barcode UMI cutoff and doublet removal based on singletCode specifications, followed by the simulation of doublets for benchmarking doublet detection methods.

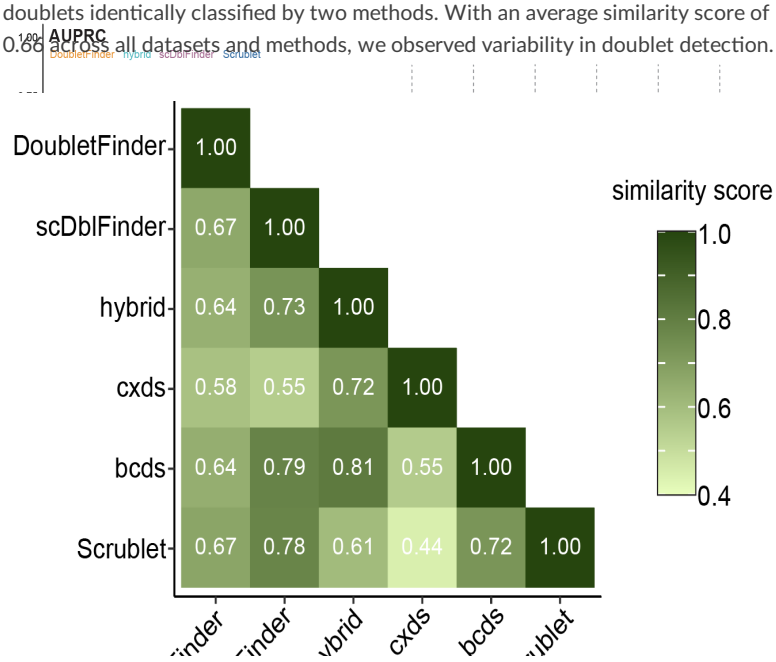
Benchmarking

We used these datasets to benchmark four doublet detection methods:

1. scDblFinder,
2. DoubletFinder,
3. Scrublet and
4. Hybrid

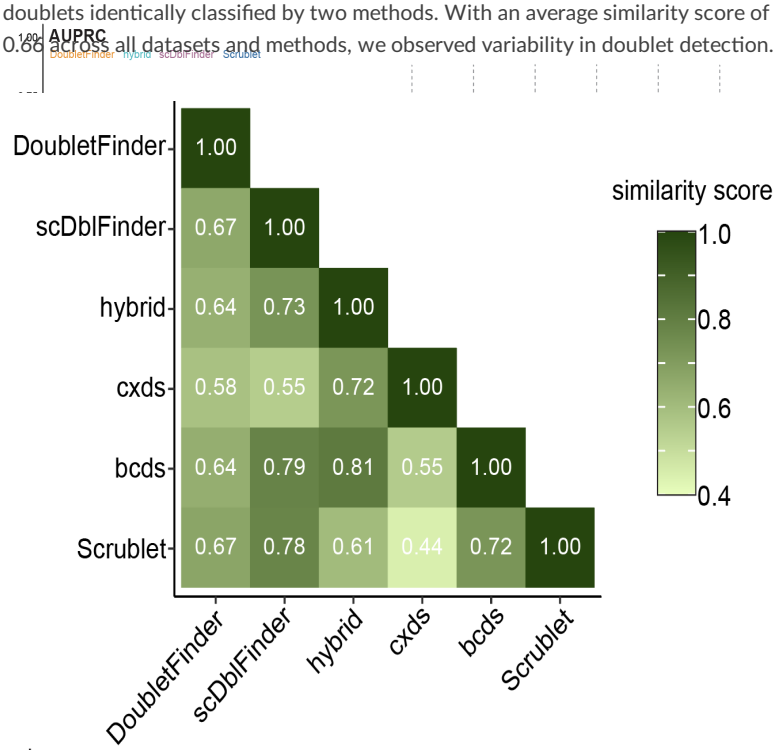
We evaluated the AUPRC, AUROC, TNR, and doublet scores and calls of the four methods and found lower than expected performance for all methods. A plot of our results for AUPRC value is found below.

We examined the consistency of doublet labeling across different doublet detection methods by introducing a 'similarity score'—a measure of the fraction of doublets identically classified by two methods. With an average similarity score of 0.66 across all datasets and methods, we observed variability in doublet detection.



methods and found lower than expected performance for all methods. A plot of our results for AUPRC value is found below.

We examined the consistency of doublet labeling across different doublet detection methods by introducing a 'similarity score'—a measure of the fraction of doublets identically classified by two methods. With an average similarity score of 0.66 across all datasets and methods, we observed variability in doublet detection.

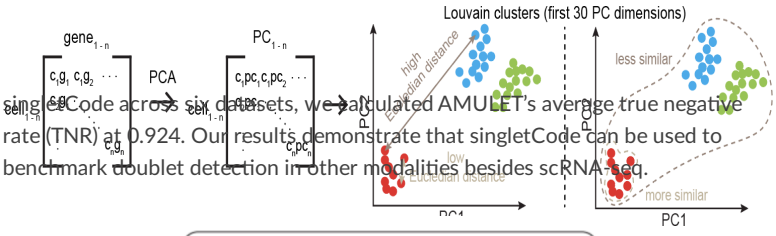


Pairwise similarity score between all benchmarked methods. A score of 1 indicates perfect identity and a 0 indicates complete disagreement. Average similarity score across all methods and datasets is 0.66.

We further evaluated doublet detection on ensemble doublet detection methods (hybrid, Chord) and across sequencing technologies (10X Genomics, Smart-seq3). For a more detailed evaluation of our results, refer to our paper.

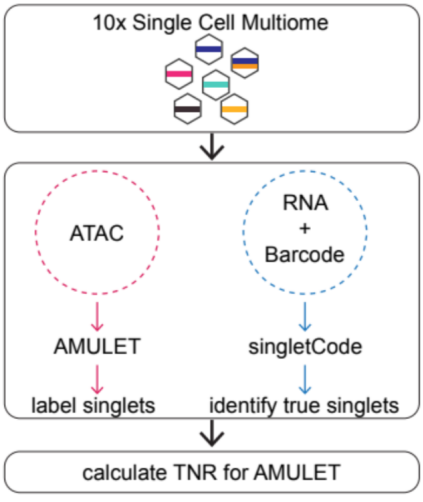
Heterogeneity effects

We wanted to know whether heterogeneity of a dataset affects the performance of doublet detection methods. Because heterogeneity can be impacted by many properties of a dataset, such as experimental design and data processing, we made conclusions based on heterogeneity within a sample. We did this by subsampling singlets and doublets within a single PC cluster for a sample (less heterogeneous, low Euclidean distance), and across all clusters for a sample (more heterogeneity, higher Euclidean distance).



sing singleCode across six datasets, we calculated AMULET's average true negative rate (TNR) at 0.924. Our results demonstrate that singletCode can be used to benchmark doublet detection in other modalities besides scRNA-seq.

Schematic of how t sampled for subsequent within- component analysis (left) was p used to identify Louvain clusters, re heterogeneous) cells are identified f tance in PC space. Transcriptionally dis from all clusters within the dataset space.

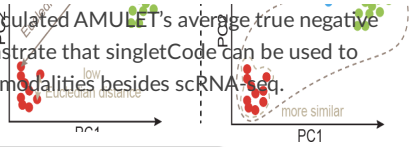


singletCode f

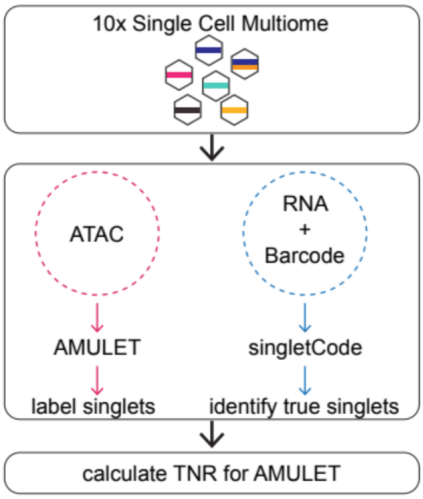
We evaluated our scATAC-seq datasets using AM erated Watermelon-barc plied AMULET to scATAC-seq fragm urrently, we identified true sing same cells. By comparing the true negative and false positive rates between AMULET and

Schematic for 10X Single Cell Multiome dataset workflow resulting in TNR calculation. The same cells in the Multiome dataset undergo scATAC-seq, scRNA-seq, and barcode sequencing. Barcodes were processed with singletCode to identify ground truth singlets.

singletCode across six datasets, we calculated AMULET's average true negative rate (TNR) at 0.924. Our results demonstrate that singletCode can be used to benchmark doublet detection in other modalities besides scRNA-seq.



Schematic of how t sampled for subsequent within- component analysis (left) was p used to identify Louvain clusters, re heterogeneous) cells are identified f tance in PC space. Transcriptionally dis from all clusters within the dataset space.



singletCode fo

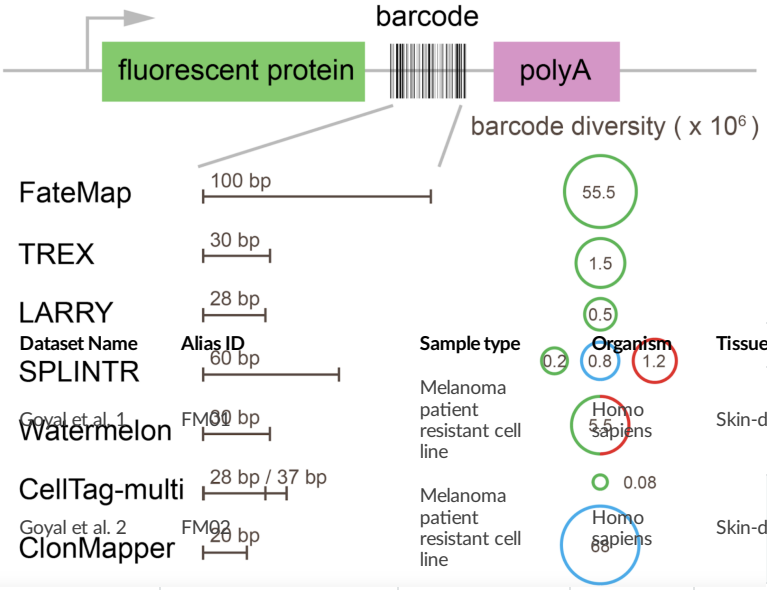
We evaluated our scATAC-seq datasets using AM erated Watermelon-barco plied AMULET to scATAC-seq fragm urrently, we identified true sing same cells. By comparing the true negative and false positive rates between AMULET and

Schematic for 10X Single Cell Multiome dataset workflow resulting in TNR calculation. The same cells in the Multiome dataset undergo scATAC-seq, scRNA-seq, and barcode sequencing. Barcodes were processed with singletCode to identify ground truth singlets while the respective fragments file from ATAC was used to label doublets with AMULET. The AMULET-labeled singlets were compared with ground truth singletCode singlets to calculate TNR.

dataset information

barcoding technologies assessed

7 different barcoding technologies We incorporated datasets using FateMap, ClonMapper, SPLINTR, LARRY, CellTag-multi, Watermelon, and TREX barcodes. Each have their own recovery and analysis specifications that we outline in Box 1 of our paper.

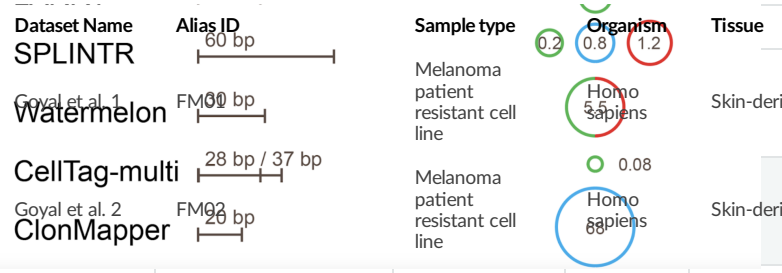


Schematic of the structure of different lineage-tracing barcodes, their length, diversity, and fluorescent colors. Each barcode contains an expressed fluorescent protein, a unique sequence, known as the barcode, and a polyA tail for capture. The lengths of barcodes in each method are shown as scale bars, and library diversities are shown as circles, sized proportional to their diversity and colored according to fluorescent proteins used.

dataset details

Specifications for barcoded and non-barcoded datasets

Dataset Name	Alias ID	Sample type	Organism	Tissue	Cell line	Genotype	Treatment	Total sam
Goyal et al. 1	FM01	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	4
Goyal et al. 2	FM02	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	4
Goyal et al. 3	FM03	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	2
Goyal et al. 4	FM04	Breast cancer patient resistant cell line	Homo sapiens	Breast; mammary gland	MDA-MB-231-D4	BRAF – G464V, KRAS – G13D, TERT – c.1-124C>T, TP53 – R280K	Chemotherapy, cytotoxic	2
Goyal et al. 5	FM05	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	2
	barcoded scRNA-seq datasets	Melanoma						



Schematic of the structure of different lineage-tracing barcodes, their length, diversity, and fluorescent color. Each barcode contains an expressed fluorescent protein, a unique sequence, known as the barcode, and a polyA tail for capture. The lengths of barcodes in each method are shown as scale bars, and library diversities are shown as circles, sized proportional to their diversity and colored according to fluorescent proteins used.

dataset details

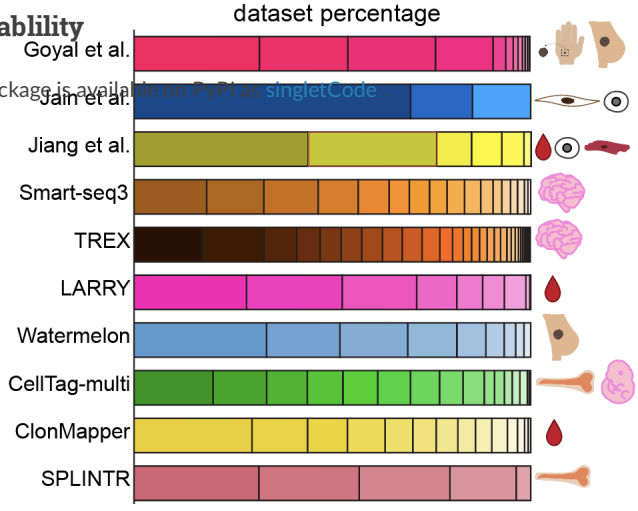
Specifications for barcoded and non-barcoded datasets

Dataset Name	Alias ID	Sample type	Organism	Tissue	Cell line	Genotype	Treatment	Total sam
Goyal et al. 1	FM01	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	4
Goyal et al. 2	FM02	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	4
Goyal et al. 3	FM03	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	2
Goyal et al. 4	FM04	Breast cancer patient resistant cell line	Homo sapiens	Breast; mammary gland	MDA-MB-231-D4	BRAF – G464V, KRAS – G13D, TERT – c.1-124C>T, TP53 – R280K	Chemotherapy, cytotoxic	2
Goyal et al. 5	FM05	Melanoma patient resistant cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	Targeted therapy	2
Goyal et al. 6	FM06	Melanoma patient untreated cell line	Homo sapiens	Skin-derived	WM989 A6-G3	BRAF – V600E	No treatment	2
Goyal et al. 8	FM08	Primary melanocytes	Homo sapiens	Skin-derived	FOM230-1	WT	No treatment	2
Jiang et al.	non_cancer	HIPS differentiation	Homo sapiens	PBMCs	PENN123i-SV20	WT	Cardiac differentiation signal	2
Jain et al.	Biorxiv	Stem cell reprogramming	Homo sapiens	Fibroblasts	hiF-T	WT	OKSM	6
TREX	TREX	neuroepithelial progenitor cells differentiation	Mus musculus	Neural	CD-1	•	No treatment	4
LARRY	LARRY, LK	pan-myeloid differentiation	Mus musculus	Blood	Lin-Kit+	•	cytokines and growth factors	18
LARRY	LARRY, LSK	pan-myeloid differentiation	Mus musculus	Blood	Lin-Kit+Sca-1+	•	cytokines and growth factors	15
SPLINTR	SPLINTR_chemo	Acute myeloid leukaemia	Mus musculus	Bone marrow	C57BL/6	MLL-AF9 + KrasG12D	chemotherapy	10
SPLINTR	SPLINTR_clone	Acute myeloid leukaemia	Mus musculus	Bone marrow	C57BL/6	MLL-AF9, MLL-AF9 + KrasG12D,	No treatment	4
Smart-seq3	Smart-seq3	neuroepithelial progenitor cells differentiation	Mus musculus	Neural	CD-1	• MLL-AF9 + Flt3ITD	No treatment	2
SPLINTR	SPLINTR_chemoretrans	Acute myeloid leukaemia	Mus musculus	Bone marrow and spleen	C57BL/6	MLL-AF9 + KrasG12D	No treatment	1
non-barcoded Watermelon	hm-12k Watermelon	Synthetic dataset	Homo sapiens, Mus musculus	Human kidney, Murine fibroblasts	HEK293T, NIH3T3	WT	No treatment	1
non-barcoded CellTag-multi	hm-6k CellTag-multi_d	Breast cancer patient resistant cell line	Homo sapiens	Breast; mammary gland	T47D	PIK3CA – H1047R; Tp53 – L194F	Targeted therapy	6
non-barcoded CellTag-multi	HMEC-orig-MULTI CellTag-multi_B4	Synthetic dataset multilineage differentiation (hematopoiesis)	Homo sapiens, Mus musculus	Human kidney, Murine bone marrow	HEK293T, NIH3T3 LSK	WT •	No treatment cytokines and growth factors	1 3
non-barcoded	HMEC-rep-MULTI	HMECs iEP reprogramming	Homo sapiens Mus musculus	mammary gland embryo	HMEC C57BL/6J	WT •	No treatment EGF	1 1
ClonMapper	ClonMapper	HMECs	Homo sapiens	mammary gland	HMEC	WT	No treatment	1
non-barcoded	HEK-HMEC-MULTI	CLL resistant cell lines Synthetic dataset	Homo sapiens Homo sapiens	blood Human kidney, mammary gland	HG3 HEK293T, HMEC	• WT	Chemotherapy No treatment	4 1
		Mouse kidney	Mus					

Dataset Name	Alias ID	Sample type	Organism	Tissue	Cell line	Genotype	Treatment	Total sam
SPLINTR	SPLINTR_clone	Acute myeloid leukaemia	Mus musculus	Bone marrow	C57BL/6	MLL-AF9 + KrasG12D,	No treatment	4
Smart-seq3	Smart-seq3	neuroepithelial progenitor cells differentiation	Mus musculus	Neural	CD-1	MLL-AF9 + Flt3ITD	No treatment	2
SPLINTR	nonbarcoded sc-RNA seq datasets SPLINTR_chemoretrans	Acute myeloid leukaemia	Mus musculus	Bone marrow and spleen	C57BL/6	MLL-AF9 + KrasG12D	No treatment	1
non-barcoded Watermelon	hm-12k Watermelon	Synthetic dataset Breast cancer patient resistant cell line	Homo sapiens, Mus musculus	Human kidney, Murine mammary fibroblasts gland	HEK293T, NIH3T3 T47D	WT PIK3CA – H1047R; Tp53 – L194F	No treatment Targeted therapy	1 6
non-barcoded CellTag-multi	hm-6k CellTag-multi_d	Synthetic multilineage dataset differentiation (hematopoiesis)	Homo sapiens, Mus musculus	Human kidney, Murine bone marrow fibroblasts	HEK293T, NIH3T3 LSK	WT	No treatment cytokines and growth factors	1 3
non-barcoded CellTag-multi	HMEC-orig-MULTI CellTag-multi_B4	HMECs iEP reprogramming	Homo sapiens Mus musculus	mammary gland embryo	HMEC C57BL/6J	WT	No treatment EGF	1 1
non-barcoded	HMEC-rep-MULTI	HMECs	Homo sapiens	mammary gland	HMEC	WT	No treatment	1
ClonMapper	ClonMapper	CLL resistant cell lines	Homo sapiens	blood	HG3		Chemotherapy	4
non-barcoded	HEK-HMEC-MULTI	Synthetic dataset	Homo sapiens	Human kidney, mammary gland	HEK293T, HMEC	WT	No treatment	1
non-barcoded	mkidney-ch	Mouse kidney cells	Mus musculus	Mouse kidney	C57BL/6J	WT	No treatment	1
non-barcoded	pbmc-2ctrl-dm	Patient PBMCs	Homo sapiens	Systemic lupus erythematosus (SLE) PBMCs	Patient-derived	Patient-derived	No treatment	1
non-barcoded	pbmc-2stim-dm	Patient PBMCs	Homo sapiens	Systemic lupus erythematosus (SLE) PBMCs	Patient-derived	Patient-derived	No treatment	1
non-barcoded	cline-ch	Human HEK, K562, KG1, and THP1	Homo sapiens	blood	human - derived	human - derived	No treatment	1
non-barcoded	pbmc-ch	Human PBMCs	Homo sapiens	blood	human - derived	human - derived	No treatment	1
non-barcoded	pdx-MULTI	Synthetic dataset	Homo sapiens, Mus musculus	Human breast cancer, mouse immue	PDX mouse model	PDX mouse model	No treatment	1
non-barcoded	nuc-MULTI	Synthetic dataset	Homo sapiens, Mus musculus	nuclei			No treatment	1
	Multiome datasets							
ATAC	Watermelon Multiome	Breast cancer patient resistant cell line	Homo sapiens	Breast; mammary gland	MCF7	GATA3 – D336G; PIK3CA – E545K	Targeted therapy	6

Stacked bar chart of cell types in all datasets. Each distinct color corresponds to one study, with the proportions of different cell types arranged from most to least prevalent (deepest to lightest) and separated by vertical lines. Adjacent to the bar chart, sketched tissues and cells provide a visualization of the cell types used in the particular study.

singletCode PyPI package



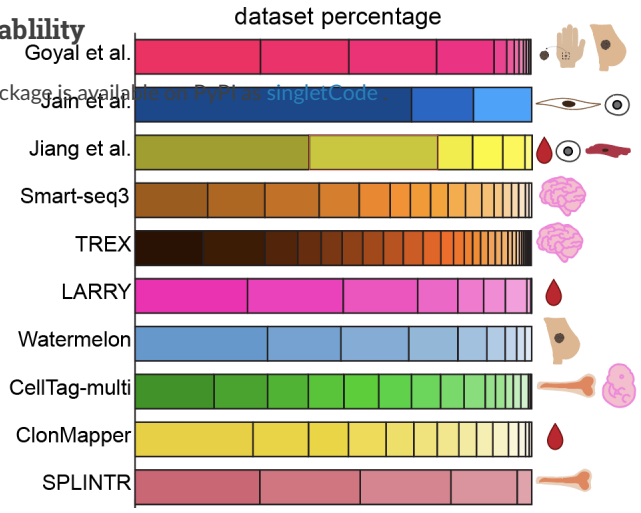
Availablility

The package is available on PyPI as singletCode.

	Multiome datasets							
ATAC	Watermelon Multiome	Breast cancer patient resistant cell line	Homo sapiens	breast; mammary gland	MCF7	GATA3 – D336G; PIK3CA – E545K	Targeted therapy	6

Stacked bar chart of cell types in all datasets. Each distinct color corresponds to one study, with the proportions of different cell types arranged from most to least prevalent (deepest to lightest) and separated by vertical lines. Adjacent to the bar chart, sketched tissues and cells provide a visualization of the cell types used in the particular study.

singletCode PyPI package



Availablility

The package is available on PyPI as singletCode .

How do I use it?

1. Installation

   It can be installed from PyPI using the following in the terminal:

```
pip3 install singletCode
from singletCode import check_sample_sheet, get_singlets
```

2. Preparing the input sample sheet.

   The input sample sheet is a .csv file that contains the information about cell ID (added while sequencing), lineage barcode, and sample name. Each row should be repeated n times where n is the number of UMIs associated with that barcode and cell ID combination. For creating the input, you can read the .csv file in as a pandas dataframe.

```
import pandas as pd
df = pd.read_csv("path/to/csv/file.csv")
```

You can check if the format and the column names are valid for running

How do I use it?

1. Installation

   It can be installed from PyPI using the following in the terminal:

   ```
   pip3 install singletCode
   from singletCode import check_sample_sheet, get_singlets
   ```

2. Preparing the input sample sheet.

   The input sample sheet is a .csv file that contains the information about cell ID (added while sequencing), lineage barcode, and sample name. Each row should be repeated n times where n is the number of UMIs associated with that barcode and cell ID combination. For creating the input, you can read the .csv file in as a pandas dataframe.

   ```
   import pandas as pd
   df = pd.read_csv("path/to/csv/file.csv")
   ```

   You can check if the format and the column names are valid for running singletCode using check_sample_sheet() function.

   ```
   check_sample_sheet(df)
   ```

   It will either return an error with information about how to modify the sample sheet to make it a valid input or will print the message, "The sample sheet provided can be used as input to get_singlets to get a list of singlets identified", in which case you can move to the next step.

3. Running get_singlets() to get an assignment of singlet status for each cell ID and barcode combination.

   This is the step where singlet identification is done using the singletCode framework. You can read more about the parameters for get_singlets() below. In this example, default values are used.

   ```
   cellLabelList, stats = get_singlets(df, dataset_name = "Sample1")
   ```

   cellLabelList is a pandas dataframe that contains 5 rows: cellID, barcode, sample, nUMI and label. Label is whether the particular cell ID and barcode combination has been called a singlet or not.

   To get a dataframe containing just the singlets, you can run this:

   ```
   singletList = cellLabelList[cellLabelList["label"] == "Singlet"]
   ```

Detailed information about the parameters for the functions in the package

get_singlets

get_singlets(*sample_sheet, output_path=None, dataset_name=None, save_all_singlet_categories=False, save_plot_umi=False, umi_cutoff_method='ratio', umi_cutoff_ratio=7.5e-06, umi_cutoff_percentile=None, min_umi_cutoff=2, umi_diff_threshold=50, umi_dominant_threshold=10*) [source]

Function that inputs the sample sheet and other parameters and runs it through count_doublets to get a list of singlets in the sample. If a row is repeated, it is assumed to reflect the UMI associated with a barcode in this cell (identified by the cellID).

Parameters

- **sample_sheet** (*DataFrame*) – A dataframe that contains 3 columns: cellID, barcode, sample.
- **dataset_name** (*str*) – The name of the dataset being analysed, it will be added to the output files and the list of cellIDs in each category of singlets if save_all_singlet_categories is set to TRUE.
- **save_all_singlet_categories** (*bool, default: False*) – If True, then the singlet categories will be saved in separate files and the column in the singlet_stats sheet returned.
- **output_path** (*str, optional*) – The path to store any output files, including plots to show UMI distribution and what the umi_cutoff used is, csv files containing singlets of different categories. If None, then the list of singlets will

total cells, total number of singlets, number of singlets recovered from different categories of singlets (such as single-barcode singlets, muli-barcode singlets, dominant-UMI singlets), number of cells removed due to low barcode UMI counts for the barcode and number of indeterminate cells since singletCode can identify only truly singlet cells but not be certain about the other cells.

You can save the singlets list (*str*) – The file and the list of cellIDs in each category of singlets, setting save_all_singlet_categories parameter to TRUE.

Detailed information about the parameters for the functions in the package

get_singlets

get_singlets(*sample_sheet, output_path=None, dataset_name=None, save_all_singlet_categories=False, save_plot_umi=False, umi_cutoff_method='ratio', umi_cutoff_ratio=7.5e-06, umi_cutoff_percentile=None, min_umi_cutoff=2, umi_diff_threshold=50, umi_dominant_threshold=10*) [source]

Function that inputs the sample sheet and other parameters and runs it through count_doublets to get a list of singlets in the sample. If a row is repeated, it is assumed to reflect the UMI associated with a barcode in this cell (identified by the cellID).

Parameters

- **sample_sheet** (*DataFrame*) – A dataframe that contains 3 columns: cellID, barcode, sample.
- **dataset_name** (*str*) – The name of the dataset being analyzed. It will be the prefix of all output files and be a column in the singlet_stats sheet returned.
- **output_path** (*str, optional*) – The path to store any output files, including plots to show UMI distribution and what the umi_cutoff used is, csv files containing singlets of different categories. If None, then the list of singlets will be returned but it won't contain information about what category of singlet each cell is. Defaults to None.
- **save_all_singlet_categories** (*bool, optional*) – If true, then singlets of each category are saved separately in csv files along with all singlets and all non-singlets. Defaults to False.
- **save_plot_umi** (*bool, optional*) – If true, then plots showing UMI distribution indicating the UMI cutoff used will be saved for each sample. Defaults to False.
- **umi_cutoff_method** (*str, optional*) – Specify if quality control for barcodes using UMI counts should be based on "ratio" or "percentile". Defaults to 'ratio'.
- **umi_cutoff_ratio** (*float, optional*) – The ratio used to determine the umi_cutoff if umi_cutoff_method is "ratio". Defaults to 3/4e5.
- **umi_cutoff_percentile** (*float, optional*) – If umi_cutoff_method is "percentile", then the umi_cutoff will be the minimum UMI count required to be in the top umi_cutoff_percentile'th percentile. There is no default and if umi_cutoff_method is set to "percentile", then manually set this parameter.
- **min_umi_cutoff** (*int, optional*) – This is the absolute minimum number of UMIs that need to be associated with a barcode for it to be considered a barcode. However, the actual umi_cutoff used will be the greater of min_umi_cutoff and the cutoff calculated using umi_cutoff_method. Defaults to 2.
- **umi_dominant_threshold** (*int, optional*) – The minimum UMI count to be associated with a barcode for it to be considered to be a potential dominant barcode in a cell. Defaults to 10.
- **umi_diff_threshold** (*int, optional*) – This is the minimum difference between UMI counts associated with a potential dominant barcode present within a cell and the median UMI count of all barcodes associated with the cell. If a cell has only one dominant barcode, it will be counted. Defaults to 50.

Return type tuple

Returns A 2-tuple containing:

- pandas.DataFrame: A dataframe which contains all unique cellID and barcode combinations in the data along with singlet assignment to each cell ID.
- pandas.DataFrame: A dataframe which contains the statistics for total singlets, different categories of singlets, and cells removed due to low UMI counts.

check_sample_sheet

check_sample_sheet(*data_frame*) [source]

Function to check if the dataframe can be used as input to get_singlets function. It checks if the three columns - cellID, barcode, sample - needed are present and in same order. If this dataframe can be used for get_singlets function, then a statement will be printed to *stdout* that

Parameters **sample_sheet** – A dataframe that contains your sample sheet.

Returns None

command line interface

Availability

The source code for the command line tool is available in our GitHub repository (repo). You can clone the repository to access the code locally:

```
git clone https://github.com/goyallab/singletcodetools/
```

and barcode combination has been called a singlet or not.

To get a dataframe containing just the singlets, you can run this:

```
singletList = cellLabelList[cellLabelList["label"] == "Singlet"]
```

total cells, total number of singlets, number of singlets recovered from different categories of singlets (such as single-barcode singlets, muli-barcode singlets, dominant-UMI singlets), number of cells removed due to low barcode UMI counts for the barcode and number of indeterminate cells since singletCode can identify only truly singlet cells but not be certain if the cell is a singlet.

You can save the list of singlets in a file and the list of cellIDs in each category of singlets by setting **save_all_singlet_categories** parameter to TRUE.

with a barcode for it to be considered a barcode. However, the actual umi_cutoff used will be the greater of min_umi_cutoff and the cutoff calculated using umi_cutoff_method. Defaults to 2.

Return type tuple

check_sample_sheet

`check_sample_sheet`(*data_frame*) [source]

Function to check if the dataframe can be used as input to get_singlets function. It checks if the sheet contains cellID, barcode, sample needed are present and in same order. If this dataframe can be used for get_singlets function, then a statement will be printed to confirm that

Parameters **sample_sheet** – A dataframe that contains your sample sheet.

Returns None

command line interface

Availability

The source code for the command line tools is available in our GitHub repository (repo). You can clone the repository to access the code locally.

```
git clone https://github.com/GoyalLab/singletCodeTools/
```

Using the interface

Navigating to commandLineTools folder, you will find 3 files and you will need to run singletCodeCommandLine.py. There are 2 modules available, one to run singletCode and the other to create a sample sheet if you have used Watermelon barcoding techmology using the fastq sequenced files from MISEQ.

Detailed information about the modules in the command line interface

singletCode command line

questions? contact us!

Do you have a doublet detection method that you would like use with singletCode? Do you have a question about how to use singletCode? Do you have a suggestion for how to improve singletCode? Please contact us!

For more information or questions, please contact Yogesh Goyal at yogesh.goyal@northwestern.edu

repositories

GitHub repository for analyses performed in our paper <https://github.com/GoyalLab/fatemap_multiplet_public>

GitHub repository for singletCode tools (command line interface and python package) <https://github.com/GoyalLab/singletCodeTools>